```
rake delete_windows_backups  # Remove files with a .bak extension
```

This section only touches on the full power of Rake. It can handle dependencies between files (for example, rebuilding an executable file if one of the source files has changed), it knows about running tests and generating documentation, and it can even package gems for you. Martin Fowler has written a good overview of Rake[2] if you're interested in digging deeper.

The Build Environment

When Ruby is compiled for a particular architecture, all the relevant settings used to build it (including the architecture of the machine on which it was compiled, compiler options, source code directory, and so on) are written to the module `RbConfig` within the library file `rbconfig.rb`. After installation, any Ruby program can use this module to get details on how Ruby was compiled:

```
require 'rbconfig'
include RbConfig
CONFIG["host"]   # => "arm64-apple-darwin22"
CONFIG["libdir"] # => "/Users/noel/.rbenv/versions/3.2.0/lib"
```

Extension libraries use this configuration file in order to compile and link properly on any given architecture.

What's Next

Now that we've seen how to run our Ruby files from the command line and use command-line and environment options within Ruby, it's time to go even wider and look at the entire Ruby Gems system for packaging tools, managing dependencies, and organizing code.

2. http://martinfowler.com/articles/rake.html